Wine Quality Predictor

Team 19

Introduction and Motivation:

The quality of the wine plays an important role when it comes to pricing, consumer perception, and production in the wine industry. However, much of the wine scoring process is normally dependent on the individual that is tasting it, who could evaluate it differently depending on things like personal preferences. In this project, we aim to develop machine-learning models that can accurately predict the quality of red wine on a scale of 1-10 (not the standard universal metric) using its physicochemical properties given in the dataset (figure 1). Our goal is to provide a reproducible method backed by data which can be used for estimating wine quality.

The models we will utilize to develop the wine score are linear regression, lasso regression, ridge regression, random forest (RF), gradient boosted trees (GBT), and neural networks (NN) for a total of 40 models, produced by varying hyperparameters for the 5 non-baseline model families. Before training the models, standard procedure would be to ensure that the data used is cleaned, split, and scaled. Following that, we will train and evaluate numerous variations of each model type, adjusting key hyperparameters such as regularization strength for linear models, layer and unit configurations for neural networks, and number of trees/depth for RF/GBT. We will evaluate using the R2 validation score, additionally using the training R2 to help determine overfitting and underfitting. After evaluating, we will identify a best performing model and test it with the test data. Theoretically, we are predicting the best model to be the RF or GBT, as they are much better at capturing nonlinear properties such as quality, which is dependent on many properties of wine. As for NN, they typically require more tuning and are more prone to instability (like overfit & underfit).

Data Description:

Our data comes from the Red Wine Quality dataset publicly available through Kaggle. This data was originally developed to study how physicochemical properties relate to how humans rated wine quality. Our dataset consisted of 1599 red wine samples of Portugal's "Vinho Verne" wine, with all its features available displayed in figure 1. The wine quality label is a sensory-based score ranging from 1 to 10, and all features in the dataset are numerical (int/float) with no missing values. It must also be noted that the distribution of quality scores is skewed toward

mid-range, so there is not a lot of data that showcase exceptionally good or poor wine. The dataset also does not include external factors such as grape fermentation or aging, meaning predictions are based solely on the given properties.

fixed acidity	volatile acidity	citric acid	residual sugar
chlorides	free sulfur dioxide	total sulfur dioxide	density
pH	sulphate	alcohol	

Figure 1 - Features in the Dataset

Methodology:

Cleanup:

Our first task was to ensure the dataset was clean. Fortunately, as mentioned, the wine dataset used contained no NaN values or missing entries that could affect or disrupt model predictions. Additionally, all the features contained relevant numerical datasets of either type float or integer, which is what we need.

Data Split and Scaling:

The data was put into a 7:2:1 split, where 70% is dedicated to training, 20% is dedicated to validating, and the remaining 10% is used for testing. After that, we standardized the inputs using StandardScaler. For consistency, it was advised to use scaled inputs to train all our models, while there exists research out there that suggests nonscaled data is better for tree based models.

Linear Regression:

For our linear regression models, the first step is to create a baseline model that contains no regularization, then followed by 5 Ridge and Lasso models each with different alpha hyperparameters. The alphas were changed in order to vary the strength of penalties applied, thus allowing us to compare differing levels of regularization. Our project evaluated the model's performance using the standard validation method, utilizing the 20% of data that was partitioned

earlier. The model quality would then be evaluated using the validation R2 score, and paired with R2 training to determine overfitting.

Random Forest:

For our random forest models, we will alter the n_estimator and max_depth hyperparameters to observe how its validation R2 differs. The hyperparameter n_estimators (number of trees) will be tested across a wide range (100-800 trees) and max_depth (tree depth) will be tested from as small as a stump (max depth of 1) to the maximum possible value. The parameter min_sample_split is going to be held constant across all 10 of our models. Model performance will be evaluated using the standard train/validation split.

Gradient Boosted Trees:

For the gradient boosted tree models, we altered the hyperparameters n_estimators and max_depth in order to test different values and the validation R2 that it produced. The hyperparameter n_estimators (number of boosting stages) will be tested across a range of 100-300 and max_depth (depth of each tree) will be tested across a range of 3-7. The hyperparameter learning_rate will be held constant in all 10 models because it does not change the architect of the model. The performance of the model was obtained using the standard train/validation method.

Neural Network:

In this model, we will build a multilayer perceptron with 3 hyperparameters and observe its resulting R2 validation score. The 3 hyperparameters are the hidden_layer (2, 3, 4, and 5), hidden_units (16, 32, 64, and 128), and the activation (relu, tanh, and sigmoid). Hyperparameters such as number of epochs and batch sizes will be held constant across all the tested models to ensure fair and controlled comparison between network architects.

Final test:

After identifying the best model and its hyperparameters based on the validation R2 score, we will use that as our final model and train it on the combined training and validation dataset (this

approach ensures the model benefits from the maximum amount of labeled data). After that, the R2 value of the final model based on test data will be evaluated.

Results and Analysis:

Model	**Best Hyperparameters**	**Training R2**	**Validation R2**
Linear Regression (Baseline)	-	0.361198	0.346165
Ridge	{'alpha': 100.0}	0.359106	0.347875
Lasso	{'alpha': 0.0001}	0.361198	0.346142
Gradient Boosted Trees	'learning_rate':0.1, 'n_estimators' : 100, 'max_depth' : 6}	0.954918	0.496065
Random Forest	{"n_estimators":800, "max_depth":None, "min_samples_split": 2}	0.927823	0.497602
MLP	{"hidden_layers":2, "hidden_units": 32, "activation": "sigmoid"}	0.416989	0.389642

Figure 2 - Summary Table

Linear Regression:

The non-regularized baseline model has resulted in a relatively small R2 value of 0.346 as displayed in figure 2. This could suggest that a linear model is not the best for the dataset. Since the training R2 is also low, the model is underfitting, which indicates that linear regression is too simple to capture the nonlinear patterns of our dataset, ultimately limiting its prediction accuracy.

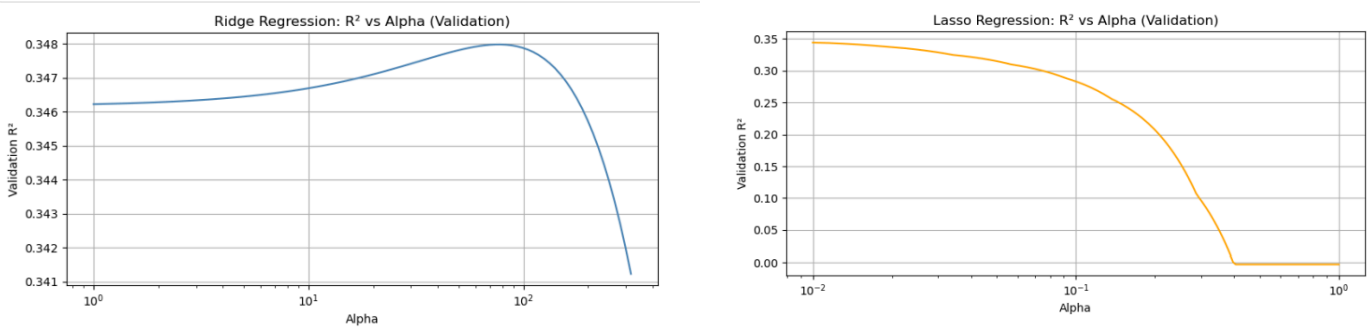

Figure 3 - This graph shows how validation R2 behaves as alpha varies in Lasso and Ridge on the logarithmic scale

Ridge and Lasso regression each yielded similar results of 0.347875 and 0.346142 R2 validation scores respectively with similar training R2. It was observed that a little regularization did help our model, but only marginally. In figure 3, the Ridge regression graph contains a hump at around 10^2 alpha, which indicates to us that a little regularization helps our model, showing the Ridge regression helps stabilize things by reducing our model's variance.. While alpha is small, the model performs similarly to the unregularized linear regression, hence suggesting the consistent R2 or of around 0.346 prior to the hump. However, if alpha gets too large our validation R2 drops significantly, suggesting that the coefficient shrank too much and the model became too simple, ultimately leading to the decline in performance.

Random Forest:

The Random Forest model was our best performing model as shown in figure 2, with a validation of 0.4976. The training R2 for this model is rather high of nearly 0.9278, which indicates that the model fits the training data well, but could also be overfitted. However, when the simpler models (more shallow models) were trained, it was observed that both the training and validation R2 performed significantly worse. This suggests that shallow nor the previously observed linear models have enough complexity to showcase the relationship in this dataset. The n_estimator hyperparameter did also result in small improvement of validation R2 while maintaining a relatively similar training R2, however it does not carry as much weight as

max_depth did, as showcased in the code with models where max_depth was held set constant at none and the number varied.

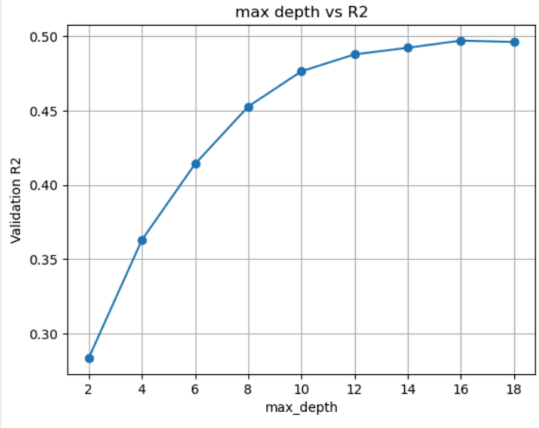

Figure 4 - Validation R2 values in Random Forest as max depth varies, n-estimator and split held constant

Figure 4 here further reinforces our point, showing how as the model becomes less shallow (higher max_depth), the validation R2 also increases until it peaks at approximately 0.5 (at around 16).

Gradient Boosted Trees:

The validation R2 scores of our GBT model was marginally worse than what the Random Forest model produced at around 0.496. The training R2 for GBT remained higher than RF, as it is sensitive to overfitting, and that each new tree is trained to correct the previous ones to fit our training data better. In our code, the one model we allowed to grow to unlimited depth created a model fitted too close to the training data, in a way failing to generalize and predict the validation data. The other models had depths hovering around 5-7, with a few lower (simpler) models that had lower depth which resulted in lower training and validation R2 (underfit).

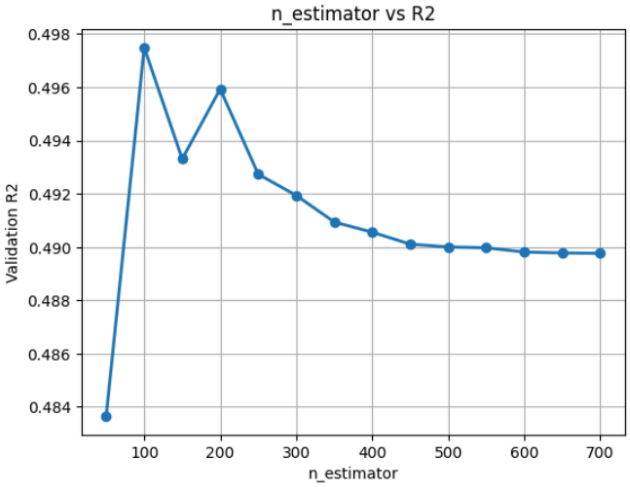

Figure 5 - This graph shows how n_estimator impacts the Validation R2 score in GBT

Aside from depth changes, we also investigated how varying the n_estimator which controlled the amount of trees impacted the validation R2 score. Shown in figure 5, using the best depth hyperparameter for GBT a graph was created to showcase how n_estimator can greatly change the validation R2 performance our model shows. It is observed that the optimal n_estimator performance peaked around 100, showcasing where the best balance between underfitting and overfitting sits. Similar to max_depth, anything beyond 200 trees reduces training error greatly, which leads to a model fit to the training set resulting in poor performance for anything else.

Neural Network:

In the 10 models that we tested, performance varied greatly, with the validation R2 falling around 0.15 to 0.38. When certain hyperparameters were tuned to achieve a higher training R2, it did not result in better validation performance, hence suggesting the models were more likely to overfit. An example would be like in our model with 3 layers and 128 units, an extremely high 0.79 training R2 was acquired yet resulted in a low validation R2 of 0.22, a case of severe overfit. Smaller networks like the one with 1 layer and 16 units, showed both low training and validation R2, hence suggesting underfitting. The optimal balance that was acquired with sigmoid activation, 2 layers, and 32 units, achieving our max validation R2 of 0.389642, which is lower than the tree models shown previously.

Final test results:

To get the final result, we concatenate together the training and validation data that was initially split, retrained our best performing model (the random forest) with the most optimal hyperparameters out of the 10 models we trained to obtain a R2 score of 0.4233.

Conclusion:

To conclude, the results in figure 2 indicate that predicting wine quality based on chemical measurements is moderately difficult. The majority of our model's validation R2 score maxed out just under 0.5, with our best performance being a Random Forest model. While linear regression and its regularized variants provided a baselines, their validation R2 values remained on the lower end, showing how our model's nonlinear data struggles to be represented. Neural networks likely performed worse than trees due to our limited training dataset size, as it would result in overfitting and overall struggle to provide the generalization for our validation data. The R2 value in GBT was close to what Random Forest had, the higher training R2 it has suggested overfitting may have deterred it from achieving the highest validation R2 shown in figure 2. Additionally, the properties we have only accounted as a factor of determining wine quality. External factors and other properties such as aging, which are capable of influencing quality outputs, are not available in the dataset.